



Space Intelligence Assistant

*Assistente Inteligente com RAG para Documentos da Nova Economia
Espacial*

Documento Técnico

Global Solution 2026

FIAP — Inteligência Artificial Generativa

1. Introdução e problema

Modelos de linguagem generativos, como os utilizados em assistentes conversacionais, respondem com base no conhecimento adquirido durante seu treinamento. Essa característica impõe uma limitação relevante: tais modelos não têm acesso a documentos específicos do usuário e, quando questionados sobre conteúdos que não fazem parte de sua base de treinamento, podem produzir respostas imprecisas ou até mesmo inventadas — fenômeno conhecido como alucinação.

No contexto da nova economia espacial, esse problema é particularmente sensível. Relatórios da NASA, dados climáticos, documentos de monitoramento ambiental e estudos sobre agricultura por satélite contêm informações técnicas e atualizadas que dificilmente estariam plenamente incorporadas ao modelo. Era necessária, portanto, uma solução capaz de consultar esses documentos diretamente antes de formular uma resposta.

O presente trabalho propõe o **Space Intelligence Assistant**, um assistente inteligente que adota a arquitetura RAG (Retrieval-Augmented Generation, ou Geração Aumentada por Recuperação) para responder perguntas a partir de documentos reais sobre o domínio espacial, fundamentando suas respostas no conteúdo recuperado e indicando os trechos utilizados como base.

2. Objetivo

O objetivo do projeto é desenvolver um assistente capaz de responder perguntas utilizando documentos relacionados à nova economia espacial — abrangendo temas como clima, satélites, agricultura inteligente, monitoramento ambiental, desastres naturais e exploração espacial. A solução deve permitir o carregamento de documentos pelo usuário, gerar representações vetoriais desses documentos, armazená-las em um banco de vetores e empregar um modelo generativo para produzir respostas contextualizadas e verificáveis.

3. Arquitetura da solução

A arquitetura segue o fluxo clássico de uma aplicação RAG, dividido em duas etapas complementares: a indexação dos documentos e a consulta. Na etapa de indexação, o documento carregado é processado uma única vez; na etapa de consulta, cada pergunta do usuário aciona a busca e a geração da resposta.

Etapa de indexação

Quando um documento é carregado, seu texto é dividido em pedaços menores, chamados *chunks*, com aproximadamente 800 caracteres cada e uma sobreposição de 150 caracteres entre eles. Essa sobreposição evita que informações importantes sejam perdidas nas fronteiras entre os pedaços. Em seguida, cada chunk é convertido em um vetor numérico — o *embedding* — que representa o significado semântico do texto. Por fim, esses vetores são armazenados e indexados no FAISS, o banco de vetores da aplicação.

Documento (PDF/TXT/DOCX) → divisão em chunks → geração de embeddings → armazenamento no FAISS

Etapa de consulta

Ao receber uma pergunta — digitada ou falada — o sistema a converte no mesmo espaço vetorial dos documentos. O FAISS então localiza os quatro trechos mais semelhantes à pergunta por meio de busca por similaridade. Esses trechos são combinados com a pergunta original e enviados ao modelo generativo, que produz a resposta fundamentada exclusivamente no conteúdo recuperado. A resposta é apresentada em texto e, opcionalmente, lida em voz alta.

Pergunta → embedding da pergunta → busca dos 4 trechos no FAISS → trechos + pergunta → modelo generativo → resposta + fontes

4. Ferramentas utilizadas

A solução foi construída com ferramentas amplamente adotadas em projetos de inteligência artificial, priorizando confiabilidade e clareza de implementação.

Ferramenta	Função no projeto
LangChain	Framework que organiza o fluxo RAG, do carregamento dos documentos à recuperação e geração
Streamlit	Construção da interface web, simples e interativa
FAISS	Banco de vetores local, responsável pelo armazenamento e pela busca por similaridade
pypdf e docx2txt	Leitura de documentos nos formatos PDF e DOCX

python-dotenv	Carregamento seguro da chave de API a partir de um arquivo de ambiente
OpenAI API	Acesso aos modelos de embeddings, geração de texto, transcrição e síntese de voz

5. Modelo utilizado

O componente generativo da aplicação utiliza o modelo `gpt-4o-mini` da OpenAI, escolhido por oferecer um bom equilíbrio entre qualidade das respostas e custo de operação. A geração dos embeddings é realizada pelo modelo `text-embedding-3-small`, responsável por transformar os trechos de texto em vetores numéricos.

Como diferencial, o projeto incorpora dois modelos adicionais voltados à interação por voz: o `whisper-1`, que transcreve a pergunta falada do usuário em texto, e o `tts-1`, que converte a resposta gerada em áudio, lendo-a em voz alta.

6. Vector store utilizada

O banco de vetores adotado foi o FAISS (Facebook AI Similarity Search), uma biblioteca de código aberto especializada em armazenar embeddings e realizar buscas por similaridade de forma eficiente. Uma vantagem importante do FAISS é que ele funciona localmente, na própria máquina, sem depender de serviços externos e sem gerar custo adicional. A busca em si é gratuita; o único custo associado ocorre no momento da geração dos embeddings, etapa de baixo custo computacional e financeiro.

7. Fluxo RAG implementado

O fluxo RAG foi implementado de modo a garantir que as respostas se mantenham fiéis ao conteúdo dos documentos. O *prompt* enviado ao modelo orienta-o explicitamente a responder apenas com base nos trechos recuperados e a informar, de maneira transparente, quando determinada informação não está presente nos documentos. Essa instrução reduz significativamente o risco de alucinações e aumenta a confiabilidade das respostas.

Além disso, a interface exibe os trechos efetivamente recuperados para cada resposta, permitindo que o usuário verifique a origem da informação. Esse recurso torna o processo de recuperação auditável e reforça a transparência da solução.

8. Diferencial: interação por voz

Para além dos requisitos obrigatórios, o assistente foi enriquecido com a capacidade de interação por voz. O usuário pode formular sua pergunta falando ao microfone; o áudio é transcrito pelo Whisper, segue pelo fluxo RAG convencional e a resposta é lida em voz alta pelo módulo de síntese de fala. O sistema também demonstra capacidade multilíngue: documentos em inglês podem ser consultados com perguntas em português, recebendo respostas em português — o que amplia consideravelmente sua aplicabilidade.

9. Limitações da solução

A solução apresenta limitações que devem ser consideradas. Em primeiro lugar, ela depende de uma chave de API da OpenAI, o que implica um custo por uso, ainda que reduzido. O banco de vetores é mantido em memória e reconstruído a cada sessão, não havendo persistência entre execuções. A qualidade das respostas está diretamente relacionada à qualidade e à relevância dos documentos carregados. Por fim, perguntas excessivamente genéricas podem não recuperar trechos úteis, uma vez que a busca é fundamentada na similaridade de conteúdo.

10. Melhorias futuras

Diversas melhorias podem ser incorporadas em versões futuras. A persistência do banco de vetores em disco evitaria o reprocessamento dos documentos a cada execução. A adição de uma etapa de *reranking* dos trechos recuperados tenderia a elevar a precisão das respostas. O uso de modelos de embeddings multilíngues e de alternativas de menor custo ampliaria a flexibilidade da solução. Por fim, a implementação de uma avaliação automática da qualidade das respostas, por meio de ferramentas como o RAGAS, permitiria medir objetivamente o desempenho do sistema.

11. Conclusão

O Space Intelligence Assistant cumpre integralmente os requisitos estabelecidos: oferece uma interface simples, permite o carregamento de documentos, gera embeddings, utiliza um banco de vetores, emprega um modelo generativo e demonstra funcionamento prático. A esses elementos soma-se o diferencial da interação por voz, na entrada e na saída. O resultado é um assistente RAG completo e funcional, capaz de responder perguntas sobre a nova economia espacial de forma fundamentada, transparente e verificável.